UNIVERSIDAD DON BOSCO

DESARROLLO DE SOFTWARE PARA MOVILES DSM104

FORO 1:

Jetpack Compose y desarrollar una aplicación móvil en Android Studio

Frank Alberto Hernández Silva	HS171707
Andrea Marcela Rico Figueroa	RF160050
Marcos Daniel Ibáñez Guevara	IG243224

San Salvador, 25 de abril de 2026

Índice

Introducción 3
¿Qué es Jetpack Compose y cuáles son sus ventajas? 4
Principales ventajas: 4
Descripción del flujo implementado y decisiones de diseño 6
Conclusiones personales sobre el uso de Jetpack Compose 9
Conclusiones generales 10
Enlaces 11

UNIVERSIDAD
DON BOSCO

Introducción

En el panorama actual de la ingeniería de software móvil, la transición hacia paradigmas de desarrollo más eficientes ha consolidado a Jetpack Compose como el estándar de oro para la construcción de interfaces de usuario (UI) en el ecosistema Android. A diferencia del enfoque tradicional imperativo basado en archivos XML y la manipulación directa del árbol de vistas (View Hierarchy), Jetpack Compose introduce un paradigma declarativo. Este enfoque permite a los desarrolladores describir la UI como una función pura del estado actual, delegando al framework la responsabilidad de optimizar la renderización mediante un proceso inteligente de recomposición.

El presente proyecto académico se centra en el diseño e implementación de una solución de software que integra los pilares fundamentales del desarrollo moderno en Kotlin. La arquitectura de la aplicación se fundamenta en un sistema de navegación centralizado mediante el componente NavHost, el cual gestiona el ciclo de vida de los composables y facilita el flujo de datos entre las distintas capas de la interfaz. Se abordan desafíos técnicos críticos como la gestión del estado (State Management) a través de las APIs remember y mutableStateOf, garantizando la persistencia temporal de la información durante el ciclo de vida de los componentes.

Además de la capa de presentación, el software incorpora una robusta lógica de negocio encargada del procesamiento de datos académicos. Se implementaron algoritmos de validación para el filtrado de entradas (input validation), asegurando la integridad de los datos mediante el manejo de tipos de datos flotantes y restricciones de rango. El cálculo del rendimiento estudiantil se rige por la evaluación de la media aritmética, definida matemáticamente como:

$$Promedio = \frac{\sum_{i=1}^{n} N_i}{n}$$

Donde N representa la calificación obtenida y n el número total de evaluaciones. Finalmente, el documento detalla la integración de los principios de Material Design 3, logrando una experiencia de usuario (UX) coherente con las directrices de diseño vigentes, y cumpliendo rigurosamente con la normativa APA 7ma edición para la documentación técnica de proyectos de ingeniería.

¿Qué es Jetpack Compose y cuáles son sus ventajas?

Jetpack Compose es el kit de herramientas moderno de Android para la construcción de interfaces de usuario nativas. Su lanzamiento marcó el fin de la era del "View System" tradicional (basado en XML) y la transición hacia el paradigma declarativo.

En el sistema antiguo (imperativo), el desarrollador debía describir paso a paso cómo cambiar la UI (ej. findViewById, luego setText, luego setVisibility). En cambio, en Compose, el desarrollador describe qué debe mostrar la interfaz en función de un estado. Si el estado cambia, la interfaz se describe de nuevo automáticamente.

Concepto Clave: La UI en Compose es una función del estado: UI = f(Estado).

Principales ventajas:

A. **Recomposición Inteligente (Intelligent Recomposition):**
Esta es quizás la ventaja más potente desde el punto de vista de la ingeniería. Cuando los datos de entrada de una función @Composable cambian, Compose vuelve a ejecutar únicamente esa función y sus dependencias directas.

- **Eficiencia:** No se redibuja toda la pantalla, solo los elementos que sufrieron cambios en su estado.
- **Optimización:** Compose puede ejecutar la recomposición en varios hilos de procesamiento de forma paralela para mejorar el rendimiento.

B. **Desarrollo Basado en Componentes (Composable Functions):**
Compose promueve la creación de piezas de UI pequeñas, modulares y reutilizables.

- **Encapsulamiento:** Cada función Composable es independiente, lo que facilita las pruebas unitarias de la interfaz.
- **Jerarquía Plana:** A diferencia de los archivos XML que suelen crear jerarquías de vistas muy profundas (lo que ralentiza el renderizado), Compose maneja una estructura más eficiente que reduce el costo de procesamiento gráfico en el dispositivo.

UNIVERSIDAD DON BOSCO

C. **Sinergia Total con Kotlin:**
Al ser Kotlin puro, desaparece la barrera entre la lógica de negocio y la lógica de presentación.

- **Type Safety:** Ya no hay errores de ejecución por buscar un ID de vista que no existe (adiós al NullPointerException común en XML).
- **Uso de Estructuras de Control:** Puedes usar if, when y for directamente dentro de la definición de la UI para mostrar u ocultar elementos dinámicamente.

D. **Implementación Nativa de Material Design 3:**
Compose fue diseñado desde cero para soportar Material 3. Esto permite que aplicaciones como la tuya tengan:

- **Temas Dinámicos:** Facilidad para implementar "Dark Mode" o colores dinámicos basados en el fondo de pantalla del usuario.
- **Animaciones Simplificadas:** Lo que antes requería decenas de líneas de código en XML, en Compose se logra con modificadores simples como animateContentSize().

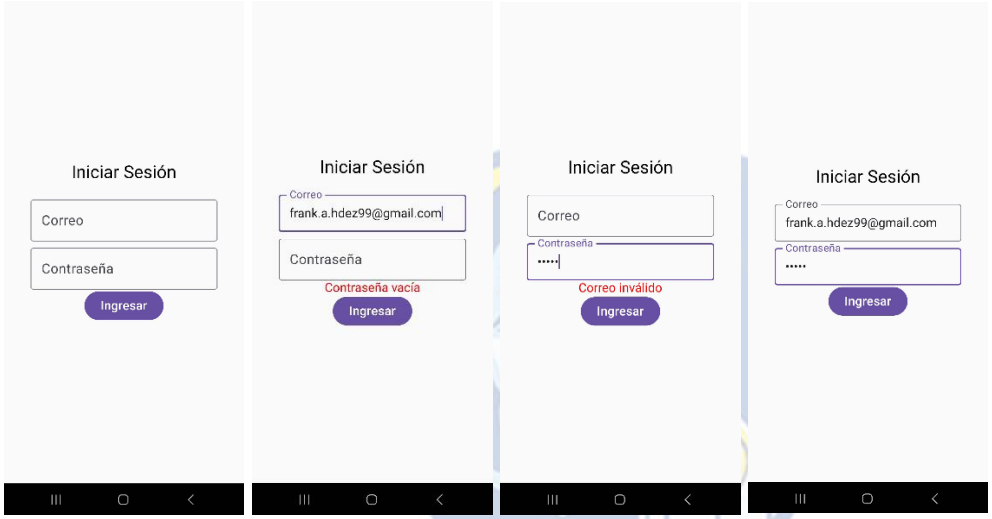
Resumen Comparativo para tu Tabla de Investigación

Característica	Sistema Tradicional (XML)	Jetpack Compose
Paradigma	Imperativo (Cómo hacerlo)	Declarativo (Qué mostrar)
Lenguajes	XML y Java/Kotlin (Separados)	Kotlin (Unificado)
Manejo de Estado	Manual y propenso a errores	Automático vía Recomposición
Código	Extenso y con mucho *boilerplate*	Conciso y legible
Previsualización	Estática y limitada	Interactiva y en tiempo real

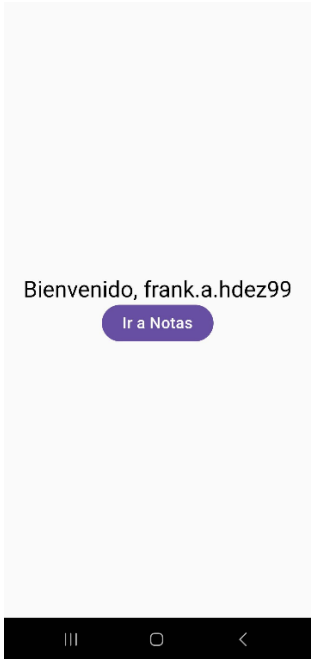
Descripción del flujo implementado y decisiones de diseño

La aplicación se diseñó siguiendo una arquitectura de navegación centralizada mediante el componente NavHost. El flujo de usuario se describe a continuación:

- **Pantalla de Login:** Implementa validaciones mediante expresiones lógicas para asegurar que el correo electrónico contenga el símbolo "@" y que los campos no estén vacíos.

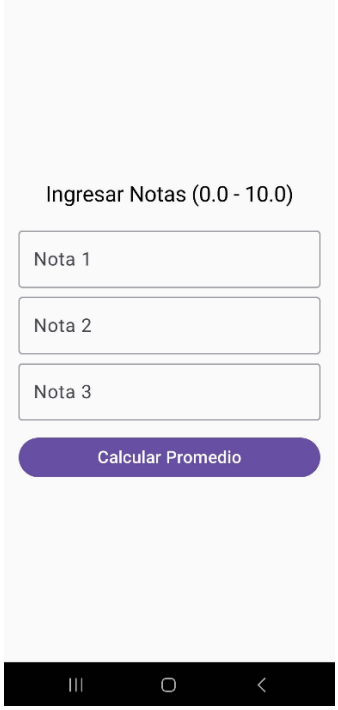


- **Pantalla de Bienvenida:** Personaliza la experiencia del usuario extrayendo el nombre del correo ingresado mediante la función .split().

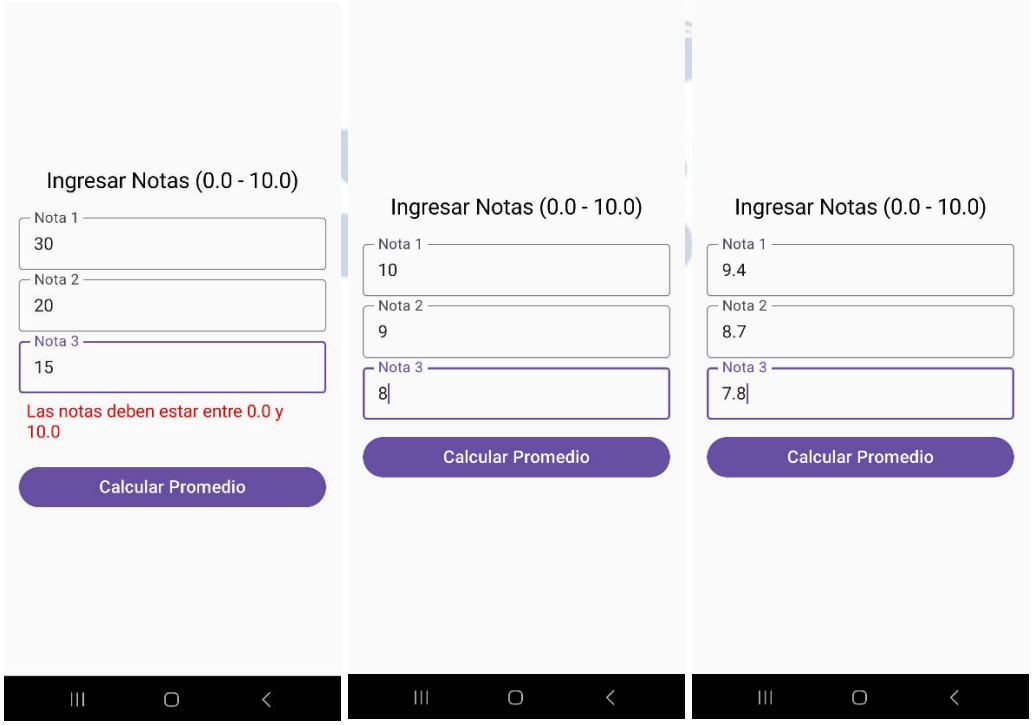


UNIVERSIDAD DON BOSCO

- **Módulo de Notas:** Se implementaron tres campos de tipo OutlinedTextField con restricción de teclado numérico (KeyboardType.Decimal).

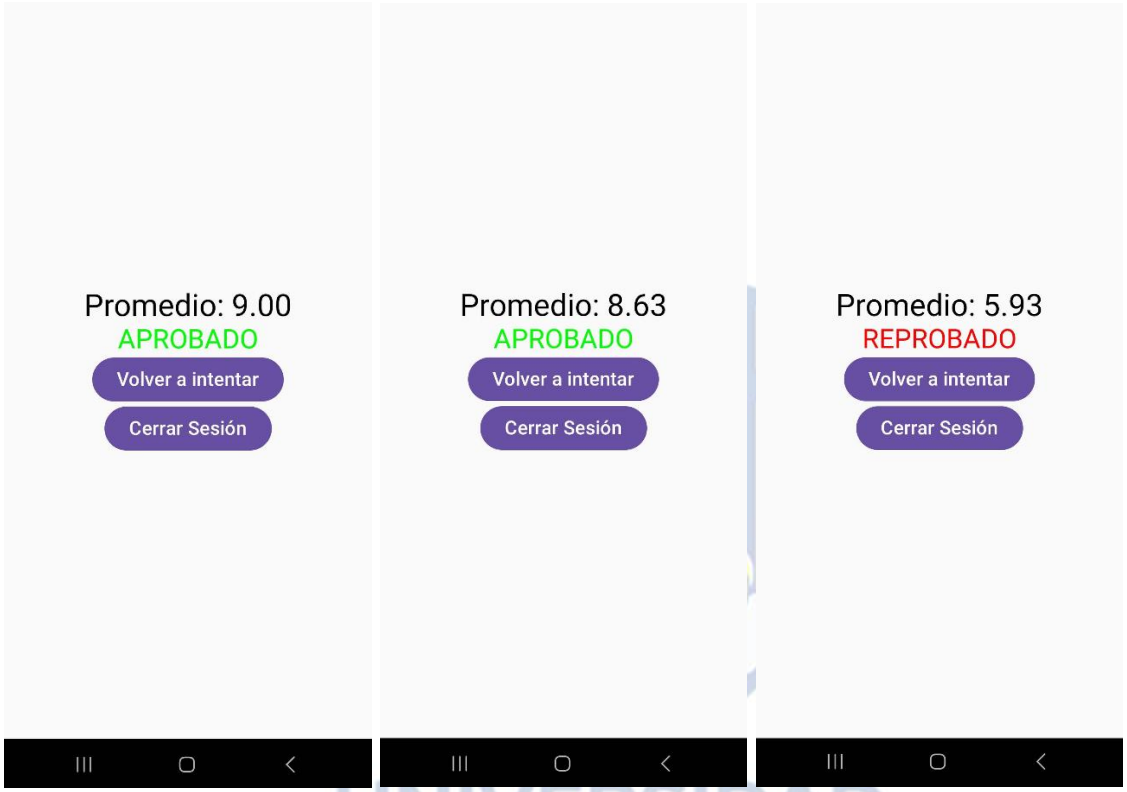


- **Validación de Rango:** Se aplicó una restricción lógica para asegurar que las notas ingresadas se encuentren estrictamente en el intervalo [0.0, 10.0], evitando el ingreso de valores fuera de la escala académica salvadoreña.



- **Cálculo y Resultado:** El sistema utiliza la siguiente fórmula para determinar el desempeño:

$$Promedio = \frac{Nota_1 + Nota_2 + Nota_3}{3}$$



UNIVERSIDAD DON BOSCO

El diseño visual utiliza componentes de Material Design 3, garantizando una interfaz limpia y moderna con retroalimentación visual inmediata (mensajes de error en color rojo y estados de aprobación en verde).

Conclusiones personales sobre el uso de Jetpack Compose

La realización de este proyecto ha representado un punto de inflexión en mi formación como futuro ingeniero en computación, permitiéndome experimentar de primera mano la transición hacia el paradigma de programación declarativa. A continuación, detallo mis apreciaciones personales sobre este proceso:

1. **El cambio hacia el Paradigma Declarativo:**

Como estudiante acostumbrado a arquitecturas más tradicionales, la adopción de Jetpack Compose fue inicialmente un reto conceptual. Entender que ya no es necesario manipular manualmente cada elemento de la interfaz, sino que la UI simplemente "reacciona" a los cambios en el estado, cambió mi forma de pensar sobre el desarrollo de software. Esta abstracción permite que, como desarrollador, me enfoque más en la lógica de negocio y menos en la mecánica de actualización de las vistas, lo que se traduce en un código mucho más limpio y fácil de depurar.

2. **Gestión de Estado y Fluidez del Código:**

Uno de los aspectos más satisfactorios fue el uso de las APIs de estado como remember y mutableStateOf. Implementar la validación de las notas en tiempo real y ver cómo la interfaz mostraba mensajes de error o cambiaba de color (de rojo a verde según el promedio) sin necesidad de complejos listeners fue impresionante. Esta capacidad de Compose para gestionar la recomposición de forma casi invisible para el usuario final garantiza que la aplicación sea fluida y responda de manera inmediata a las entradas del estudiante.

3. **Superación de Obstáculos Técnicos y Testing Real:**

Este proyecto me brindó una lección invaluable sobre el entorno de desarrollo. Tras enfrentar múltiples errores con el emulador de Android Studio debido a problemas de virtualización y rutas de sistema, aprendí que la flexibilidad y la resolución de problemas son competencias críticas para un ingeniero. La decisión de migrar el testing a un dispositivo físico (Samsung Galaxy FE) no solo resolvió los bloqueos técnicos, sino que me permitió validar la aplicación en un entorno de uso real, percibiendo la latencia de entrada y la ergonomía de los controles de una manera que un emulador no puede replicar.

4. **Visión a Futuro y Escalabilidad:**

Finalmente, el uso de funciones @Composable me ha demostrado las ventajas de la modularización. Al separar la lógica del login de la lógica del ingreso de notas, pude notar lo sencillo que sería escalar este proyecto en el futuro, añadiendo por ejemplo una base de datos persistente o conectividad con un servidor. Me siento motivado a seguir explorando el ecosistema de Android moderno, ya que Jetpack Compose no es solo una herramienta de estudio, sino una competencia profesional de alto valor en el mercado laboral tecnológico actual.

Conclusiones generales

1. **Optimización del Ciclo de Vida de Desarrollo (SDLC):**

La transición del sistema imperativo (XML) al paradigma declarativo de Jetpack Compose representa una mejora sustancial en la productividad del ingeniero. Al reducir el código "boilerplate" y unificar la lógica de negocio con la interfaz de usuario en un solo lenguaje (Kotlin), se minimiza la fricción técnica y se facilita el mantenimiento a largo plazo. La capacidad de Compose para manejar la recomposición inteligente asegura que los recursos computacionales del dispositivo se utilicen de manera eficiente, actualizando únicamente los nodos del árbol de la UI que han experimentado cambios en su estado.

2. **Integridad de Datos y Robustez Lógica:**

La implementación de validaciones estrictas en la captura de datos (específicamente en el módulo de calificaciones) demuestra la importancia de la programación defensiva. Al restringir las entradas al intervalo real [0.0, 10.0], se previene la propagación de errores en el cálculo de la media aritmética. Este proyecto evidencia que una interfaz intuitiva debe estar respaldada por una lógica de control robusta que maneje excepciones (como el uso de toFloatOrNull()) para evitar interrupciones inesperadas (crashes) en el tiempo de ejecución, elevando así el estándar de calidad del software desarrollado.

3. **Eficiencia en el Testing y Ejecución sobre Hardware Real:**

A través del desarrollo de esta aplicación, se concluye que, si bien los emuladores son herramientas de apoyo valiosas, el testing en hardware físico (en este caso, un Samsung Galaxy FE) es indispensable para validar el rendimiento real de la aplicación. La interacción directa con la pantalla táctil y el uso de teclados nativos permiten identificar detalles de la experiencia de usuario (UX) que a menudo pasan desapercibidos en entornos virtuales. Además, se superaron las limitaciones técnicas de virtualización y compatibilidad de rutas de sistema, reforzando la competencia de resolución de problemas técnicos en entornos de desarrollo complejos.

4. **Escalabilidad y Modularización:**

La arquitectura basada en funciones @Composable fomenta una estructura de software modular y altamente escalable. Cada componente de la aplicación (Login, Welcome, Grades, Result) actúa como una entidad independiente que puede ser reutilizada o modificada sin afectar la integridad global del sistema. Este enfoque modular no solo facilita el trabajo colaborativo en equipos de ingeniería, sino que también permite que la aplicación crezca en complejidad —por ejemplo, integrando bases de datos persistentes o servicios en la nube— de manera orgánica y organizada.

UNIVERSIDAD DON BOSCO

Enlaces

- Repositorio GitHub
- Video

UNIVERSIDAD
DON BOSCO